

网络技术与应用实验报告

实验二：利用NPcap编程捕获数据包

姓名：谢小珂 学号：2310422 专业：物联网工程

一、实验要求

IP数据报捕获与分析编程实验，要求如下：

- (1) 了解NPcap的架构。
- (2) 学习NPcap的设备列表获取方法、网卡设备打开方法，以及数据包捕获方法。
- (3) 通过NPcap编程，实现本机的IP数据报捕获，计算捕获IP数据报的校验和，并与数据报的校验和字段比较。
- (4) 捕获的数据报不要求硬盘存储，但应以简单明了的方式在屏幕上显示。必显字段包括源MAC地址、目的MAC地址、源IP、目的IP地址、校验和字段的数值、程序计算出的校验和数值。
- (5) 编写的程序应结构清晰，具有较好的可读性。

二、实验环境与配置

2.1 NPcap / NPcap SDK 安装

为实现数据包捕获，需要安装 **NPcap 驱动**与 **NPcap SDK**：

1. 访问 NPcap 官网并下载最新版本安装包

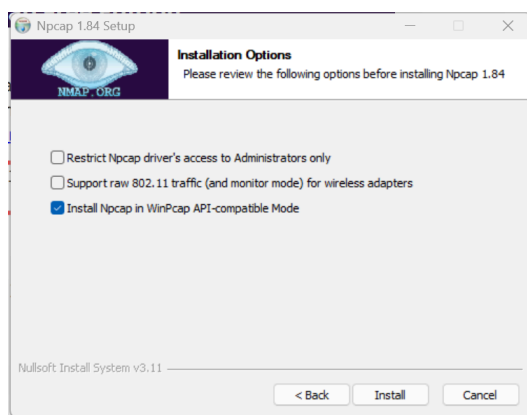
Downloading and Installing Npcap Free Edition

The free version of Npcap may be used (but not externally redistributed) on up to 5 systems ([free license details](#)). It may also be used on unlimited systems where it is only used with [Nmap](#), [Wireshark](#), and/or [Microsoft Defender for Identity](#). Simply run the executable installer. The full source code for each release is available, and developers can build their apps against the SDK. The improvements for each release are documented in the [Npcap Changelog](#).

- [Npcap 1.84 installer](#) for Windows 7/2008R2, 8/2012, 8.1/2012R2, 10/2016, 2019, 11 (x86, x64, and ARM64).
- [Npcap SDK 1.15](#) (ZIP).
- [Npcap 1.84 debug symbols](#) (ZIP).
- [Npcap 1.84 source code](#) (ZIP).

The latest development source is in our [Github source repository](#). Windows XP and earlier are not supported; you can use [WinPcap](#) for these versions.

2. 安装时勾选：**Install Npcap in WinPcap API-compatible Mode**

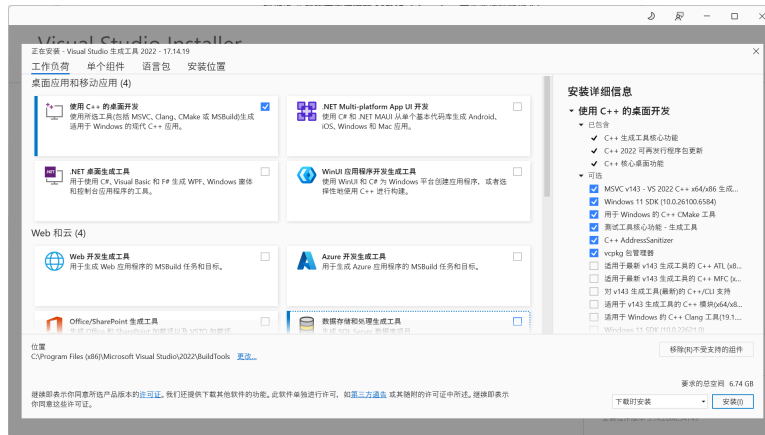


3. 下载 NPcap SDK，解压后可看到两个关键目录：

- | | | |
|---|---------|-----------------------|
| 1 | Include | → 头文件目录 |
| 2 | Lib | → 库文件目录 (x64 / win32) |

2.2 组件与安装

- 编辑器: Visual Studio Code
- 编译器: Microsoft Visual C++ (MSVC), 通过 Visual Studio Build Tools 安装
 - 工作负载: 使用 C++ 的桌面开发 (含 MSVC v143、Windows SDK、CMake)



- 抓包驱动与 SDK: Npcap (运行时) + Npcap SDK (开发头/库)
- 说明: 本实验选用 x86 (32 位) 编译目标, 与当前 `wpcap.lib` / `Packet.lib` 的库架构一致, 避免 “x86/x64 不匹配” 的链接错误。

安装完成后, 将 Npcap SDK 中的 **Include** 与 **Lib** 目录拷贝到工程根目录, 形成本地依赖, 避免全局路径配置。

2.3 Npcap SDK 配置 (不配置环境变量, 直接放项目目录)

将 Npcap SDK 解压后, 把里面的 **Include** 和 **Lib** 文件夹直接放到实验工程目录:

- | | | |
|---|-------------|------------------------------------|
| 1 | IP_capture/ | |
| 2 | main.cpp | ← 实验代码 |
| 3 | | |
| 4 | Include/ | ← SDK 头文件 (pcap.h) |
| 5 | Lib/ | ← SDK 库文件 (wpcap.lib / Packet.lib) |

优点: 路径不用写绝对路径, 也不用手动配置系统环境变量。

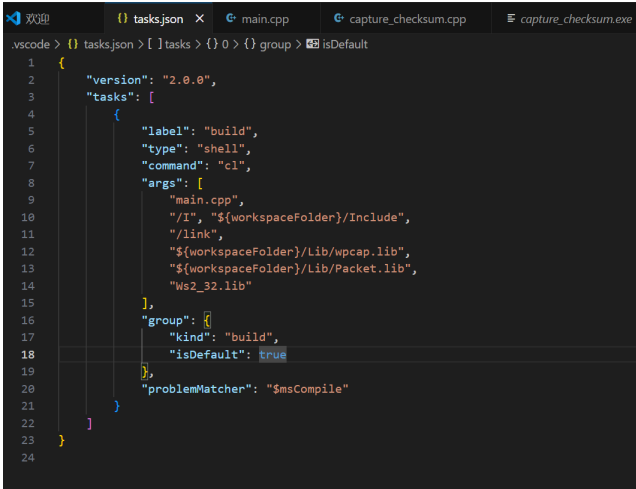
 capture_checksum.cpp	2025/10/30 9:29	C++ Source
 main.cpp	2025/10/30 0:33	C++ Source
 main.exe	2025/10/30 0:31	应用程序
 main.obj	2025/10/30 0:31	VisualStudio.obj.d1...
 .vscode	2025/10/30 0:25	文件夹
 Include	2025/10/30 0:21	文件夹
 Lib	2025/10/30 0:21	文件夹

2.4 VS Code 配置编译任务（使用 MSVC）

在项目中新建：

```
1 | .vscode/tasks.json
```

内容如下：



运行编译： `Ctrl + Shift + B`

输出：

```
1 | Microsoft (R) C/C++ Optimizing Compiler
2 | main.cpp
3 | Microsoft (R) Incremental Linker
4 | out: capture_checksum.exe
```

终端运行：

```
1 | .\capture_checksum.exe
```

三、实验过程

本实验的目标是：通过 Npcap 编程实现对本机网络接口的原始数据包捕获，并自行解析 IP 头部，验证 IP 校验和的正确性。

整个原理可以理解为：

Npcap = 把“Wireshark看到的原始包”搬到 C/C++ 代码里。

3.1 Npcap的架构

Npcap 在 Windows 上由 三层架构 组成：内核驱动层 → 用户态库（Packet.dll / wpcap.dll）→ 应用层（本实验 capture_checksum.cpp）。

(1) 内核层：NPF.sys（与 Windows NDIS 协议栈交互）

Npcap 的内核模块名称为 NPF.sys，挂载在 Windows 的 NDIS 网络协议栈上，负责：

- 截获流经网卡的 **原始以太网帧**
- 处理混杂模式（允许接收非本机 MAC 的帧）
- 写入内核缓冲区供用户态读取

打开网卡的操作对应 Npcap 源码 `PacketOpenAdapter()`：

在 **Packet32.h** 中可以看到适配器结构体 `ADAPTER`，其成员包含 `hFile`（驱动句柄）：

```
1 typedef struct _ADAPTER
2 {
3     HANDLE hFile;    ///< Handle to an open instance of the NPF driver.
4     ...
5 } ADAPTER, * LPADAPTER;
```

说明：NPF.sys 作为驱动，应用最终通过 `hFile` 与内核通信，读取数据包。

(2) 用户态库：Packet.dll + wpcap.dll（封装为 pcap API 提供给程序使用）

DLL	作用
Packet.dll	通过 DeviceIoControl 与 NPF.sys 通信，负责打开网卡、读写数据
wpcap.dll (libpcap for Windows)	提供标准 pcap API： <code>pcap_findalldevs</code> / <code>pcap_open_live</code> / <code>pcap_loop</code>

设备枚举 → `PacketGetAdapterNames()`

在 Packet32.h 可看到：

```
1  BOOLEAN PacketGetAdapterNames(_Out_ PCHAR pStr, _Inout_ PULONG BufferSize);
```

`pcap_findalldevs()` 内部就是调用 `PacketGetAdapterNames()` 获取网卡列表。

打开网卡 → `PacketOpenAdapter()`

源码声明：

```
1  LPADAPTER PacketOpenAdapter(PCCH AdapterName);
```

`pcap_open_live()` 实际调用的就是 `PacketOpenAdapter`。

内核读包 → `PacketReceivePacket()`

```
1  BOOLEAN PacketReceivePacket(LPADAPTER AdapterObject, LPPACKET lpPacket, BOOLEAN Sync);
```

这是 Npcap 把内核缓冲区数据复制到用户态缓冲区的关键函数。

过滤器 BPF（在内核执行）

`pcap_compile` / `pcap_setfilter` 会把 BPF 设置给驱动。

pcap-bpf.h 中定义了 BPF 过滤指令结构：

```
1 | struct bpf_program { ... };
```

若设置 "ip"，则非 IP 数据在内核丢弃，减少 CPU 消耗。

(3) 应用层：capture_checksum.cpp（本实验的程序，实现报文解析与校验）

实验写的 capture_checksum.cpp 与 Npcap 的三层架构的对应关系如下：

NPcap 层	调用 / 源码证据	本实验代码
用户态库 (wpcap.dll)	pcap_findalldevs() 调用 PacketGetAdapterNames	列出网卡编号供选择
用户态库 → 驱动	pcap_open_live() 调 PacketOpenAdapter	打开设备，进入混杂模式
驱动 → 应用层回调	pcap_loop() 调 PacketReceivePacket	触发 packet_handler()

代码（片段）：

```
1 | pcap_open_live(d->name, 65536, 1, 1000, errbuf);
2 | pcap_loop(handle, -1, packet_handler, NULL);
```

每次抓到数据包，PacketReceivePacket → pcap_loop → 调用 packet_handler()。

NPcap 数据流



3.2设备列表获取方法

在 Npcap 环境下，本实验通过 **libpcap API** 获取本机可用于抓包的网卡接口列表。核心函数是：

```
1 | pcap_findalldevs()
```

如图所示：

```
==== Available Network Interfaces ====
[1] \Device\NPF_{18BC3F02-EF24-4F2F-A87E-A073D378153E} (WAN Miniport (Network Monitor))
[2] \Device\NPF_{90900E2D-CCF5-4BE6-8264-F981FDE24E77} (WAN Miniport (IPv6))
[3] \Device\NPF_{EE8086DD-B42D-43B0-81BB-95A61623712D} (WAN Miniport (IP))
[4] \Device\NPF_{8B917CD9-4CDA-4FEB-9841-4F078EE63ACE} (Hyper-V Virtual Ethernet Adapter)
[5] \Device\NPF_{492E4F9A-C958-4DEA-978F-1E7B36DBDF2E} (Bluetooth Device (Personal Area Network))
[6] \Device\NPF_{22664115-B708-43FE-A277-EBCAB31D0C18} (Intel(R) Wi-Fi 6 AX201 160MHz)
[7] \Device\NPF_{A7A6AF5-0217-4064-8B7F-695554F1A7E0} (VMware Virtual Ethernet Adapter for VMnet8)
[8] \Device\NPF_{C566F866-7953-4C9E-8712-CD186A84FFEF} (VMware Virtual Ethernet Adapter for VMnet1)
[9] \Device\NPF_{8399018D-74F5-418D-B239-D8B410B0D86D} (Microsoft Wi-Fi Direct Virtual Adapter #2)
[10] \Device\NPF_{B8F704D8-1167-4157-9A81-F1F52DE67FC2} (Microsoft Wi-Fi Direct Virtual Adapter)
[11] \Device\NPF_{5D6AEB40-3A46-4FCA-B939-E3F319EFA4F1} (Famatech Radmin VPN Ethernet Adapter)
[12] \Device\NPF_{Loopback} (Adapter for loopback traffic capture)
[13] \Device\NPF_{3128A90D-141E-4803-AAB5-85947099FA25} (Realtek PCIe GbE Family Controller)
[14] \Device\NPF_{EEF2BB7A-78AB-40BD-9233-8E9B29F3E2EB} (Sangfor SSL VPN CS Support System VNIC)

Select interface index: |
```

(1) 获取网卡设备列表 (pcap_findalldevs)

程序首先声明 `pcap_if_t *alldevs`，用于存放 Npcap 返回的链表结构。

```
1 | pcap_if_t *alldevs;
2 | char errbuf[PCAP_ERRBUF_SIZE];
3 |
4 | if (pcap_findalldevs(&alldevs, errbuf) == -1) {
5 |     printf("Error in pcap_findalldevs: %s\n", errbuf);
6 |     return -1;
7 | }
```

解释

- `pcap_findalldevs()` 会向 Npcap 的用户态库 `wpcap.dll` 请求 **设备列表链表**。
- 失败返回 `-1`，错误信息写入 `errbuf`。

(2) 遍历设备链表，输出给用户选择

使用 `for` 循环遍历 `pcap_if_t` 链表，每个设备对应一个网卡接口。

```
1 | int i = 0;
2 | for (d = alldevs; d != NULL; d = d->next) {
3 |     printf("[%d] %s (%s)\n", ++i, d->name,
4 |         d->description ? d->description : "No description");
5 | }
```

解释

- `d->name` 是设备标识（通常为 `\Device\NPF_{GUID}`），来自 NPF 驱动层。

- `d->description` 是网卡的可读名称，如 `Intel(R) Wi-Fi 6 AX201`。
- 程序将设备编号展示给用户选择。

运行程序时的输出内容：

```
1  ==== Available Network Interfaces ====
2  [6] \Device\NPF_{22664115-B708-43FE-A277-XXXXXX} (Intel(R) Wi-Fi 6 AX201)
3  ...
```

(3) 用户选择一个设备，用于后续 `pcap_open_live` 打开

```
1  printf("\nselect interface index: ");
2  int index;
3  scanf("%d", &index);
4
5  for (i = 1, d = alldevs; d != NULL && i < index; d = d->next, i++);
```

解释

- 让用户输入编号（例如 `6`）。
- 程序根据编号定位到对应链表节点 `d`，作为后续抓包设备。

(4) 清理内存（释放链表）

```
1  pcap_freealldevs(alldevs);
```

解释

- `pcap_findalldevs()` 使用动态内存，因此需主动释放，否则会造成内存泄漏。

3.3 网卡设备打开方法

(1) 总体流程

在 `capture_checksum.cpp` 中，打开网卡设备并开始捕获的核心流程位于 `main()`：

1. 使用 `pcap_findalldevs()` 获取设备链表（见 3.2 已写部分）；
2. 用户选择设备编号并从链表定位对应 `pcap_if_t * d`（代码位置见下）；
3. 调用 `pcap_open_live()` 打开所选设备句柄 `pcap_t *handle`；
4. 编译并安装 BPF 过滤器以提高效率；
5. 启动捕获循环 `pcap_loop()` 并在 `packet_handler()` 中处理每个数据包；捕获结束后调用 `pcap_close()` 释放资源。

这些步骤在源码中对应的位置为：设备打开与检查 `pcap_open_live` → 过滤器设置 → 捕获循环与关闭。

(2) pcap_open_live() 的参数与含义

```
1  pcap_t *handle = pcap_open_live(d->name, 65536, 1, 1000, errbuf);
```

各参数语义：

- `d->name`：要打开的设备名（`pcap_findalldevs` 返回的 `\Device\NPF_{GUID}`），见设备列表构建处。
- `65536` (`snaplen`)：抓包最大字节数（本程序取 `65536`，确保抓取完整以太网帧与 IP 选项，避免头部被截断）。
- `1` (`promisc`)：设置为混杂模式（`promiscuous=1`），以接收网卡能看到的所有流经帧，而不仅限于发往本机的帧。
- `1000` (`read timeout`，毫秒)：读/缓冲超时，影响用户态从内核读取数据的时机（本程序设置为 `1000 ms`）。
- `errbuf`：错误缓冲区，用于接收 `pcap_open_live` 的错误消息（若返回 `NULL`，程序打印并退出）。

源码的错误处理：若 `pcap_open_live` 返回 `NULL`，程序会打印错误并释放 `alldevs` 后退出（见 `pcap_open_live` 后的错误处理分支）。

(3) 安装 BPF 过滤器

为了减小用户态处理压力，程序在打开设备后尝试编译并安装 BPF 过滤器 `"ip"`（仅捕获 IPv4 包）：

```
1  struct bpf_program fp;
2  if (pcap_compile(handle, &fp, "ip", 1, PCAP_NETMASK_UNKNOWN) == 0) {
3      pcap_setfilter(handle, &fp);
4      pcap_freecode(&fp);
5  }
```

- `pcap_compile`：把人类可读的过滤表达式编译成 BPF 程序（如果成功返回 0）；编译后 `fp` 持有字节码。
- `pcap_setfilter`：把编译好的 BPF 字节码安装到捕获句柄上；Npcap 会尽可能在内核级或驱动附近应用过滤，从而减少上送到用户态的包数。
- `pcap_freecode`：释放 `pcap_compile` 分配的资源。

注意：本程序选择 `"ip"` 过滤器是因为后续仅对 IPv4 报文（并计算 IP header checksum）感兴趣；这一点在 `packet_handler()` 中以 `eth_type == 0x0800` 的逻辑再次确认。

(4) 启动捕获循环与关闭设备

程序使用 `pcap_loop` 启动无限循环捕获，回调函数为 `packet_handler`：

```
1  pcap_loop(handle, -1, packet_handler, NULL);
2  pcap_close(handle);
```

- `pcap_loop(handle, -1, ...)`：-1 表示无限捕获直到程序被中断（例如 Ctrl+C）；当捕获到每个包时，libpcap 会调用 `packet_handler()` 并传入 `pcap_pkthdr` 与帧字节指针。
- `pcap_close(handle)`：捕获结束后释放 `pcap_t` 句柄与相关资源。

(5) 与 `packet_handler()` 的配合（为何要用 65536 / promisc）

`packet_handler()` 假定接收到的是完整以太网帧并从中解析 IPv4 头（以太网 14 字节偏移开始），读取 IP 头的 `ihl` 字段并按字节读取 header 用于校验和计算（见 `packet_handler` 中对 `ip`、`ihl` 的处理）。为保证这些解析可靠：

- **snaplen 必须足够大**（否则 IP 头或选项可能被截断），代码使用 65536 作为安全值。
- **混杂模式（promisc=1）** 允许程序看到除本机外的流量（有助于观察更多样本），因此代码选择打开混杂模式。

小结

- `pcap_open_live()` 是程序与 Npcap 驱动建立数据通道的入口；本程序通过 `pcap_open_live(d->name, 65536, 1, 1000, errbuf)` 打开设备，保证抓全帧、开启混杂模式并以 1000ms 的超时读取数据。
- 成功打开后，程序可编译/安装 BPF 过滤器（这里是 "ip"）以减少用户态回调。
- 使用 `pcap_loop()` 启动无限捕获，回调 `packet_handler()` 对以太网帧与 IP 头做解析与校验和比较；捕获结束后调用 `pcap_close()` 释放资源。

3.4 数据包捕获方法

(1) 启动抓包循环（`pcap_loop`）

打开网卡后，程序使用 `pcap_loop()` 启动捕获：

```
1 | pcap_loop(handle, -1, packet_handler, NULL); // 无限循环抓包
```

- `-1` 表示无限抓包直到用户按 Ctrl+C
- 每抓到一个数据包，libpcap/Npcap 会自动回调 `packet_handler()`，将捕获的以太网帧送给程序处理。

(2) `packet_handler`：接收并处理数据包

回调定义：

```
1 | void packet_handler(u_char *user,
2 |                     const struct pcap_pkthdr *h,
3 |                     const u_char *bytes)
```

Npcap 每捕获到一帧以太网数据，就会把：

参数	内容
<code>h</code>	报文头，含时间戳、长度等
<code>bytes</code>	完整以太网帧数据起始地址

传入该函数。

(3) 最小长度检查 (避免越界)

```
1 | if (h->len < 14 + 20) return;
```

- 以太网头固定 **14 字节**
- IPv4 固定头最少 **20 字节**
- 若不足说明不是完整 IP 数据包，直接丢弃。

(4) 解析以太网头，判断是否为 IPv4

```
1 | const u_char *eth = bytes;  
2 | uint16_t eth_type = (eth[12] << 8) | eth[13];  
3 | if (eth_type != 0x0800) return;
```

- `bytes[12..13]` 是 EtherType
- `0x0800` 表示 IPv4，否则直接返回

(5) 定位 IP 头，并读取 IHL (首部长度)

```
1 | const u_char *ip = bytes + 14;           // 跳过以太网头  
2 | int ihl = ip[0] & 0x0F;  
3 | int ihl_bytes = ihl * 4;  
4 | if (ihl_bytes < 20) return;
```

- `ip[0]` 高 4 位 = 版本号，低 4 位 = IHL (单位：4 字节)
- 转换成真实字节长度 `ihl * 4`
- 若小于 20 字节，则为非法 IP 头，丢弃

(6) 取出 IP 头内的校验和字段

```
1 | uint16_t checksum_in = (ip[10] << 8) | ip[11];
```

- IP 校验和字段固定在偏移 `10` 和 `11` 字节

(7) 调用程序实现的 checksum 计算函数

```
1 | uint16_t checksum_calc = compute_ip_checksum(ip, ihl_bytes);
```

校验函数定义在同文件：

```

1  for (int i = 0; i < ihl_bytes; i += 2) {
2      if (i == 10) continue;          // 跳过 checksum 字段
3      uint16_t word = (hdr[i] << 8) | hdr[i + 1];
4      sum += word;
5  }

```

核心行为：

1. 按 16-bit 累加 IP 头所有字段
2. 跳过原始校验和字段
3. 溢出回卷
4. 按 RFC 标准取反

(8) 输出 MAC / IP / 校验和对比

```

1  printf("Dest MAC: "); print_mac(eth + 0);
2  printf("Src  MAC: "); print_mac(eth + 6);
3
4  sprintf_s(src_ip, "%u.%u.%u.%u", ip[12], ip[13], ip[14], ip[15]);
5  sprintf_s(dst_ip, "%u.%u.%u.%u", ip[16], ip[17], ip[18], ip[19]);
6
7  printf("IP header checksum (in packet): 0x%04X\n", checksum_in);
8  printf("IP header checksum (calc):      0x%04X\n", checksum_calc);

```

输出结果示例：

```

1  IP header checksum (in packet): 0x0000
2  IP header checksum (calc):      0x996E
3  Checksum: FAIL

```

3.5完整代码与注释

NPcap + C/C++ 抓包 & 校验和验证程序功能总结：

1. 调用 pcap_findalldevs() 获取本机全部网卡设备。
2. 用户选择网卡，使用 pcap_open_live() 打开该设备进行抓包。
3. 设置 BPF 过滤规则，仅捕获 IPv4 数据包（提高效率）。
4. pcap_loop() 持续抓包，捕获到每个数据包后调用 packet_handler() 回调处理。
5. 在 packet_handler() 中：
 - 解析以太网帧头（目的 MAC / 源 MAC）
 - 判断是否为 IPv4 报文 (EtherType == 0x0800)
 - 读取 IP Header，提取 IP header checksum 字段
 - 调用 compute_ip_checksum() 重新计算 IP 校验和

- 将报文中的校验和与程序计算的校验和比较，输出结果

```
1  #include <winsock2.h>
2  #include <ws2tcpip.h>
3  #include <pcap.h>
4  #include <stdio.h>
5  #include <stdint.h>
6
7  // 指定链接依赖库（保证 CL/cl.exe 编译不会找不到库）
8  #pragma comment(lib, "wpcap.lib")
9  #pragma comment(lib, "Packet.lib")
10 #pragma comment(lib, "ws2_32.lib")
11
12 //
13 *****
14
15 // print_mac() — 打印 MAC 地址（6 字节）
16 //
17 *****
18
19 void print_mac(const u_char *mac) {
20     printf("%02X:%02X:%02X:%02X:%02X:%02X",
21           mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
22 }
23
24 //
25 *****
26
27 // compute_ip_checksum() — 计算 IPv4 头部校验和
28 // 参数:
29 //     hdr      : 指向 IP Header 起始位置
30 //     ihl_bytes : IP 头长度（单位: 字节），由 IHL 字段 * 4 得到
31 // 算法依据: RFC 791 IPv4 Header Checksum
32 //
33 *****
34
35 uint16_t compute_ip_checksum(const u_char *hdr, int ihl_bytes) {
36     uint32_t sum = 0;
37
38     // 按 16-bit 两两累加 IP 头内容
39     // 关键: 校验和字段（第 10、11 字节）必须视为 0
40     for (int i = 0; i < ihl_bytes; i += 2) {
41         if (i == 10) continue; // 跳过 checksum 原字段
42         uint16_t word = (hdr[i] << 8) | hdr[i + 1];
43         sum += (uint32_t)word;
44     }
45
46     // 如果字节数奇数（理论上不会出现），则补齐处理
47     if (ihl_bytes % 2 == 1) {
48         sum += (uint16_t)(hdr[ihl_bytes - 1] << 8);
49     }
50
51     // 把高位溢出部分加回低 16 位（回卷）
```

```

44     while (sum >> 16) {
45         sum = (sum & 0xFFFF) + (sum >> 16);
46     }
47
48     // 最后取反，得到最终校验和
49     return (~sum) & 0xFFFF;
50 }
51
52 //
53 ****
54 // packet_handler() -- Npcap 抓到每一个数据包后都会进入该回调
55 // 参数:
56 //     bytes : 完整的“以太网帧”数据，从 MAC 开始
57 //     h      : pcap 捕获到的数据包头（含长度等）
58 //
59 ****
60 void packet_handler(u_char *user, const struct pcap_pkthdr *h, const u_char *bytes)
61 {
62     (void)user;          // 未使用该参数（抑制 VS 警告）
63
64     // 判断是否足够长: 14 字节以太网头 + 20 字节最小 IP Header
65     if (h->len < 14 + 20) return;
66
67     const u_char *eth = bytes;          // 以太网头起始地址
68     uint16_t eth_type = (eth[12] << 8) | eth[13];
69
70     // 判断是否为 IPv4 (EtherType == 0x0800)
71     if (eth_type != 0x0800) return;
72
73     // 定位到 IP Header (MAC 头后第 14 个字节开始)
74     const u_char *ip = bytes + 14;
75
76     // IP Version (高 4 位) + IHL (低 4 位, 单位 4B)
77     int ihl = ip[0] & 0x0F;
78     int ihl_bytes = ihl * 4;
79     if (ihl_bytes < 20) return;          // 非法 IP Header
80
81     // 报文中原本的校验和字段
82     uint16_t checksum_in = (ip[10] << 8) | ip[11];
83
84     // 程序计算所得 checksum
85     uint16_t checksum_calc = compute_ip_checksum(ip, ihl_bytes);
86
87     // ===== 信息输出 =====
88     printf("---- Packet captured (len=%u) ----\n", (unsigned)h->len);
89
90     printf("Dest MAC: "); print_mac(eth);      printf("\n");
91     printf("Src  MAC: "); print_mac(eth + 6);  printf("\n");
92
93     // 提取 IP 源地址与目的地址
94     char src_ip[INET_ADDRSTRLEN] = {0};

```

```

92     char dst_ip[INET_ADDRSTRLEN] = {0};
93     sprintf_s(src_ip, sizeof(src_ip), "%u.%u.%u.%u", ip[12], ip[13], ip[14],
ip[15]);
94     sprintf_s(dst_ip, sizeof(dst_ip), "%u.%u.%u.%u", ip[16], ip[17], ip[18],
ip[19]);
95
96     printf("Src IP: %s\n", src_ip);
97     printf("Dst IP: %s\n", dst_ip);
98     printf("Header IHL: %d bytes\n", ihl_bytes);
99
100    printf("IP header checksum (in packet): 0x%04X\n", checksum_in);
101    printf("IP header checksum (calc):      0x%04X\n", checksum_calc);
102
103    // 校验结果判断
104    if (checksum_in == checksum_calc)
105        printf("Checksum: OK\n\n");
106    else
107        printf("Checksum: FAIL\n\n");    // 很多 fail 是因为网卡硬件做了 checksum
offload (重要!)
108 }
109
110 //
*****
****
111 // main() -- 程序入口
112 //
*****
****
113 int main() {
114     char errbuf[PCAP_ERRBUF_SIZE] = {0};
115     pcap_if_t *alldevs = NULL;
116
117     // ① 获取本机所有网卡设备
118     if (pcap_findalldevs(&alldevs, errbuf) == -1) {
119         printf("pcap_findalldevs failed: %s\n", errbuf);
120         return 1;
121     }
122
123     printf("==== Available Network Interfaces ==== \n");
124
125     // ② 遍历打印所有网卡名称 (如 \Device\NPF_{xxx})
126     int index = 0;
127     pcap_if_t *d;
128     for (d = alldevs; d; d = d->next) {
129         printf("[%d] %s (%s)\n",
130             ++index,
131             d->name,
132             d->description ? d->description : "No description");
133     }
134
135     if (index == 0) {
136         printf("No interfaces found.\n");
137         return 1;

```

```

138     }
139
140     // ③ 用户输入网卡编号
141     int choice;
142     printf("\nSelect interface index: ");
143     scanf_s("%d", &choice);
144
145     index = 0;
146     for (d = alldevs; d; d = d->next) {
147         if (++index == choice) break;
148     }
149
150     if (!d) {
151         printf("Invalid choice.\n");
152         pcap_freealldevs(alldevs);
153         return 1;
154     }
155
156     // ④ 打开该网卡进行抓包
157     printf("\nOpening device: %s (%s)\n",
158           d->name,
159           d->description ? d->description : "No desc");
160
161     // pcap_open_live 参数解释:
162     // - d->name          → 网卡设备名称
163     // - 65536            → 最大抓包长度 (最大帧大小)
164     // - 1                → 混杂模式 (能够捕获所有流经的数据包)
165     // - 1000 ms          → 读取超时
166     pcap_t *handle = pcap_open_live(d->name, 65536, 1, 1000, errbuf);
167     if (!handle) {
168         printf("pcap_open_live failed: %s\n", errbuf);
169         pcap_freealldevs(alldevs);
170         return 1;
171     }
172
173     pcap_freealldevs(alldevs);
174
175     // ⑤ 设置 BPF 过滤规则: 仅捕获 "IPv4"
176     struct bpf_program fp;
177     if (pcap_compile(handle, &fp, "ip", 1, PCAP_NETMASK_UNKNOWN) == 0) {
178         pcap_setfilter(handle, &fp);
179         pcap_freecode(&fp);
180     }
181
182     printf("\nStart capturing... (Press Ctrl+C to stop)\n\n");
183
184     // ⑥ 开始抓包! 每捕获一个包就会进入 packet_handler()
185     pcap_loop(handle, -1, packet_handler, NULL);
186
187     pcap_close(handle);
188     return 0;
189 }

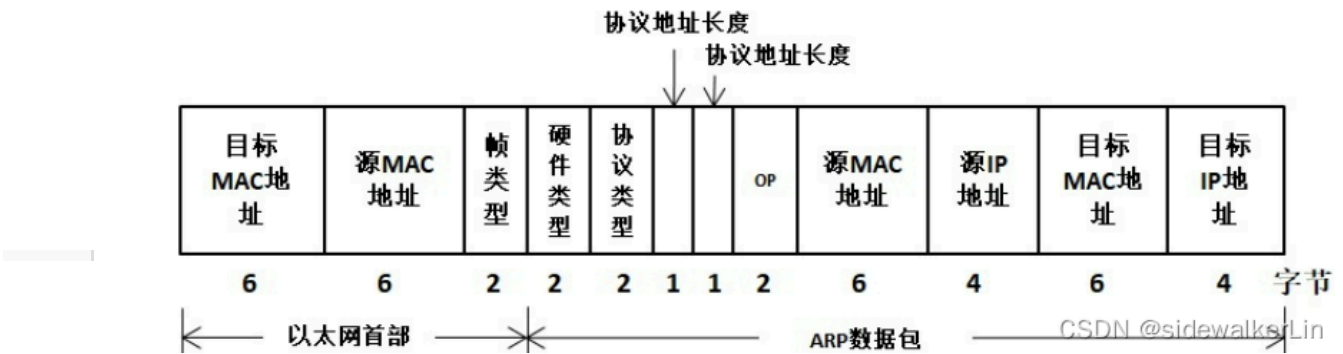
```

3.6结果分析

捕获到的数据报首先包含 **以太网首部 (Ethernet Header)**，随后根据 EtherType 字段进一步判断协议类型（如 0x0800 表示 IPv4，0x0806 表示 ARP）。

如下图所示（ARP 数据帧结构），以太网帧头固定为 **14 字节**，其中包含：

- 目标 MAC 地址 (6B)
- 源 MAC 地址 (6B)
- 类型 Type 字段 (2B):
 - 0x0800 → IPv4 数据报
 - 0x0806 → ARP 数据包



在本实验中，程序使用 `pcap_loop()` 捕获数据帧，并在 `packet_handler()` 回调中读取以太网帧头：

```
1 | const u_char *eth = bytes;
2 | uint16_t eth_type = (eth[12] << 8) | eth[13];
```

当 `eth_type == 0x0800` 时，即表示这是 **IPv4 数据报**，随后程序进一步定位到 IP 首部（跳过 14 字节以太网头），并计算：

- IP Header Checksum（计算校验和并与报文内字段比较）

因此，这张 ARP 帧结构图很好地说明了：

程序解析数据包时是从以太网头开始，逐层递进最终获取到 IP 头部并验证校验和。

```
---- Packet captured (len=60) ----
Dest MAC: 74:3A:F4:03:3D:88
Src MAC: 00:00:5E:00:01:FD
Src IP: 10.137.144.40
Dst IP: 10.136.69.227
Header IHL: 20 bytes
IP header checksum (in packet): 0x14C3
IP header checksum (calc): 0x14C3
Checksum: OK

---- Packet captured (len=586) ----
Dest MAC: FF:FF:FF:FF:FF:FF
Src MAC: 7A:B1:75:CE:3E:DC
Src IP: 10.136.124.202
Dst IP: 10.136.127.255
Header IHL: 20 bytes
IP header checksum (in packet): 0x9B35
IP header checksum (calc): 0x9B35
Checksum: OK

---- Packet captured (len=60) ----
Dest MAC: 74:3A:F4:03:3D:88
Src MAC: 00:00:5E:00:01:FD
Src IP: 10.137.144.40
Dst IP: 10.136.69.227
Header IHL: 20 bytes
IP header checksum (in packet): 0x3E5E
IP header checksum (calc): 0x3E5E
Checksum: OK

---- Packet captured (len=60) ----
Dest MAC: 00:00:5E:00:01:FD
Src MAC: 74:3A:F4:03:3D:88
```


第一组包（示例片段）：

```
1 | ---- Packet captured (len=60) ----
```

- `Packet captured`：表示程序在 `pcap_loop()` 的一次回调中收到了一个数据包（由 `packet_handler()` 产生输出）。
- `(len=60)`： `h->len` 值 —— 原始包长度（字节数）。这来自 `pcap_pkthdr` 的 `len` 字段（或 `caplen`），显示这个以太网帧的总字节数。用于判断是否足够解析（代码有最小长度检查 `if (h->len < 14 + 20)` `return;`）。

```
1 | Dest MAC: 74:3A:F4:03:3D:88
```

- `Dest MAC:`：标签，随后跟着的是目的 MAC 地址（以太网帧头的前 6 字节）。
- `74:3A:F4:03:3D:88`：目的 MAC 的六个字节，十六进制表示，按规范由高位到低位展示（每字节两位十六进制）。在以太网中这是包要送到的下一跳（本例是某一主机的物理地址）。

```
1 | Src MAC: 00:00:5E:00:01:FD
```

- `Src MAC:`：标签，表示源 MAC（以太网帧头的 6 字节源地址）。
- `00:00:5E:00:01:FD`：源 MAC 地址的十六进制表示 —— 说明该帧来自这个物理网卡或虚拟网卡。

```
1 | Src IP: 10.137.144.40
```

- `Src IP:`：标签，表示 IP 层源地址（在 IP 头偏移 12..15 字节）。程序用 `sprintf_s(src_ip, ..., ip[12], ip[13], ip[14], ip[15])` 把四字节格式化为点分十进制。
- `10.137.144.40`：IP 层源地址，表示此 IP 数据报的发送者。

```
1 | Dst IP: 10.136.69.227
```

- `Dst IP:`：标签，IP 层目的地址（IP 头偏移 16..19）。

```
1 | Header IHL: 20 bytes
```

- `Header`：指 IP 首部（header）。
- `IHL:`：IP Header Length 的简称（IHL = Internet Header Length）。它在 IP 头第 0 个字节的低 4 位中存储，单位是 32-bit words（4 字节）。代码取 `int ihl = ip[0] & 0x0F; int ihl_bytes = ihl * 4;`。
- `20 bytes`：当 IHL = 5（最常见）时，`ihl_bytes = 5*4 = 20`，说明该 IP 头没有选项字段，只有最小长度的 20 字节。

```
1 | IP header checksum (in packet): 0x14C3
```

- `IP header checksum (in packet):`: 标签，表示“报文里（wire）IP头部的 checksum 字段的原始值”。该字段就是 IP 头偏移 10-11 的 16-bit 值（网络字节序）。程序用 `uint16_t checksum_in = (ip[10] << 8) | ip[11]`；读取并打印。
- `0x14C3`：十六进制显示的原始校验和值。这是发送端在发包时放在头部的值（若硬件卸载存在，可能为 0）。

```
1 | IP header checksum (calc):      0x14C3
```

- `IP header checksum (calc):`: 标签，表示程序根据抓到的 IP 头（将校验字段视为 0）重新计算得到的校验和。对应代码 `uint16_t checksum_calc = compute_ip_checksum(ip, ihl_bytes)`。

校验和计算实现（`compute_ip_checksum`）

◦ 算法概述：

- 按 16-bit 大端字取值把 IP 头（除校验字段）全部累加到 `uint32_t sum`。
 - 若 `ihl_bytes` 为奇数（通常不会发生），对最后一字节按高字节补 0 处理。
 - 把高 16 位回卷加到低 16 位直到无高位（`while (sum >> 16) sum = (sum & 0xFFFF) + (sum >> 16);`）。
 - 最后取反（`~sum`）& `0xFFFF` 得到校验和。
- `0x14C3`：计算结果，与报文内字段一致（说明头部在发送时已经填写了正确的 checksum，或抓到的包是入站/没有卸载问题）。

```
1 | Checksum: OK
```

- `Checksum:`: 标签，表示比较结果。
- `OK`: 说明 `checksum_in == checksum_calc`，即报文内的校验和和程序计算的校验和匹配。

常见输出情形对照说明（为什么有时是 OK，有时是 FAIL）

- `Checksum: OK`: 抓到的包里 `ip[10..11]` 字段已经含有正确的校验和值，或抓到的是入站数据（网卡已完成校验）；程序的 `compute_ip_checksum` 按 RFC 算法得到相同结果，因此显示 OK。
- `Checksum: FAIL`: 很多网卡在发送路径启用了 **Checksum Offloading**，操作系统把 IP 头的 checksum 字段留成 0 或不最终填值，网卡在实际发送到线路之前由硬件计算并写入。Npcap 很可能截获的是内核提交给网卡但网卡尚未写入 checksum 的缓冲副本（因此 `checksum_in == 0x0000`），但程序按头部内容计算出非零 checksum，于是显示 FAIL。

四、实验总结

在本次实验中，通过使用 Npcap 捕获原始以太网帧并解析 IP 首部，我对链路层与网络层之间的关系有了直观、深入的理解，特别是亲手实现 IP Header 校验和计算，使我不再停留在书本概念，而是真正理解了“校验和只作用于 IP 头”以及硬件 checksum offloading 导致抓包中 checksum 字段可能为 0 的现象。

此外，代码仍具有改进空间，例如：加入对 ARP 等其他协议的识别、在检测到硬件卸载导致校验和为 0 时给出提示、支持将捕获的数据存储为 .pcap 文件供 Wireshark 分析等，这些都会让程序更加健壮和实用。